Docker Unified UIMA Interface (DUUI)

A framework at the dawn of a new era for NLP processing

Automatic analysis of large text corpora is a complex task. This complexity particularly concerns the question of time efficiency. Furthermore, efficient, flexible, and extensible textanalysis requires the continuous integration of every new text analysis tools. Since there are currently, in the area of NLP and especially in the application context of UIMA, only very few to no adequate frameworks for these purposes, which are not simultaneously outdated or can no longer be used for security reasons, this work will present a new approach to fill this gap. To this end, we present Docker Unified UIMA Interface (DUUI), a scalable, flexible, lightweight, and featurerich framework for automated and distributed analysis of text corpora that leverages experience in Big Data analytics and virtualization with Docker.

TLDR

Javadoc

Tutorials

- **Simple Sentiment**
- **Advance Hate Check**
- **Complex Fact Check**

Kubernetes-Setup

Features

Using DUUI, NLP preprocessing on texts can be performed using the following features:

Horizontal and vertical scaling
Capturing heterogeneous annotation landscapes
Capturing heterogeneous implementation landscapes
Reproducible & reusable annotations
Monitoring and error-reporting
Lightweight usability

Functions

DUUI has different components which are distinguished into Drivers and Components.

Components

Components represent the actual analysis methods for recognizing (among others) tokens, named entities, POS and other ingredients of the NLP. All components must be analysis methods in the definition of UIMA. Of course, existing analysis methods based on Java can also be used directly (e.g. dkpro).

Independently of this, Components can also be implemented in alternative programming languages, as long as the interface of DUUI is used, they can be targeted and used.

Driver

DUUI has a variety of drivers that enable communication as well as the execution of Components in different runtime environments.

UIMADriver

The UIMADriver runs a UIMA Analysis Engine (AE) on the local machine (using local memory and processor) in the same process within the JRE and allows scaling on that machine by replicating the underlying Analysis Engine. This enables the use of all previous analysis methods based on UIMA AE without further adjustments.

DockerDriver

The DUUI core driver runs Components on the local Docker daemon and enables machine-specific resource management. This requires that the AEs are available as Docker images according to DUUI to run as Docker containers. It is not relevant whether the Docker image is stored locally or in a remote registry, since the Docker container is built on startup. This makes it very easy to test new AEs (as local containers) before being released. The distinction between local and remote Docker images is achieved by the URI of the Docker image used

RemoteDriver

AEs that are not available as containers and whose models can or should not be shared can still be used if they are available via REST. Since DUUI communicates via RESTful, remote endpoints can be used for pre-processing. In general, AEs implemented based on DUUI can be accessed and used via REST, but the scaling is limited regarding request and processing capabilities of the hosting system. In addition, Components addressed via the RemoteDRiver can be used as services. This has advantages for AEs that need to hold large models in memory and thus require a long startup time. To avoid continuous reloading, it may be necessary to start a service once or twice in a dedicated mode and then use a RemoteDriver to access it. To use services, their URL must be specified to enable horizontal scaling.

SwarmDriver

The SwarmDriver complements the DockerDriver; it uses the same function alities, but its AEs are used as Docker images distributed within the Docker Swarm network. A swarm consists of n nodes and is controlled by a leader node within the Docker framework. However, if an application using DUUI is executed on a Docker leader node, the individual AEs can be

executed on multiple swarm nodes.

KubernetesDriver

The KubernetesDriver works similarly to the SwarmDriver, but Kubernetes is used as the runtime environment instead of Docker Swarm.

Requirements

Java 17 Docker 22.10

UIMA-Components

A list of existing DUUI components as Docker images can be found here.

> **Note**
>
> Instructions for creating your own DUUI components and detailed explanations can be found under Tutorials.

Using

There are basically two ways to use DUUI for preprocessing texts:

Clone the GitHub project.
Include the GitHub project using JitPack via maven (Recommended).

Using JitPack

Add the following to your pom file:

```
<repositories>
  <repository>
      <id>jitpack.io</id>
      <url>https://jitpack.io</url>
  </repository>
</repositories>
```

After that DUUI can be integrated as a dependency using JitPack 1.5.5

```
<dependency>
  <groupId>com.github.texttechnologylab</groupId>
  <artifactId>DockerUnifiedUIMAInterface</artifactId>
  <version>1.4</version>
</dependency>
```

Use with Java

```
int iWorkers = 2; // define the number of workers

JCas jc = JCasFactory.createJCas(); // A empty CAS document is defined.

// load content into jc ...

// Defining LUA-Context for communication
DUUILuaContext ctx = LuaConsts.getJSON();

// Defining a storage backend based on SQlite.
DUUISqliteStorageBackend sqlite = new DUUISqliteStorageBackend("loggingSQlite.db")
            .withConnectionPoolSize(iWorkers);

// The composer is defined and initialized with a standard Lua context as well with a storage
DUUIComposer composer = new DUUIComposer().withLuaContext(ctx)
                        .withScale(iWorkers).withStorageBackend(sqlite);

// Instantiate drivers with options (example)
DUUIDockerDriver docker_driver = new DUUIDockerDriver()
        .withTimeout(10000);
DUUIRemoteDriver remote_driver = new DUUIRemoteDriver(10000);
DUUIUIMADriver uima_driver = new DUUIUIMADriver().withDebug(true);
DUUISwarmDriver swarm_driver = new DUUISwarmDriver();

// A driver must be added before components can be added for it in the composer. After that th
composer.addDriver(docker_driver, remote_driver, uima_driver, swarm_driver);

// A new component for the composer is added
composer.add(new DUUIDockerDriver.
    Component("docker.texttechnologylab.org/gnfinder:latest")
    .withScale(iWorkers)
    // The image is reloaded and fetched, regardless of whether it already exists locally (opt
    .withImageFetching());

// Adding a UIMA annotator for writing the result of the pipeline as XMI files.
composer.add(new DUUIUIMADriver.Component(
                createEngineDescription(XmiWriter.class,
                        XmiWriter.PARAM_TARGET_LOCATION, sOutputPath,
```

```
                )).withScale(iWorkers));

// The document is processed through the pipeline. In addition, files of entire repositories c
composer.run(jc);
```

> ℹ Note
>
> Further examples can be found at the tutorials.

Cite

If you want to use the project please quote this as follows:

Alexander Leonhardt, Giuseppe Abrami, Daniel Baumartz and Alexander Mehler. (2023). “Unlocking the Heterogeneous Landscape of Big Data NLP with DUUI.” Findings of the Association for Computational Linguistics: EMNLP 2023, 385–399. [LINK] [PDF]

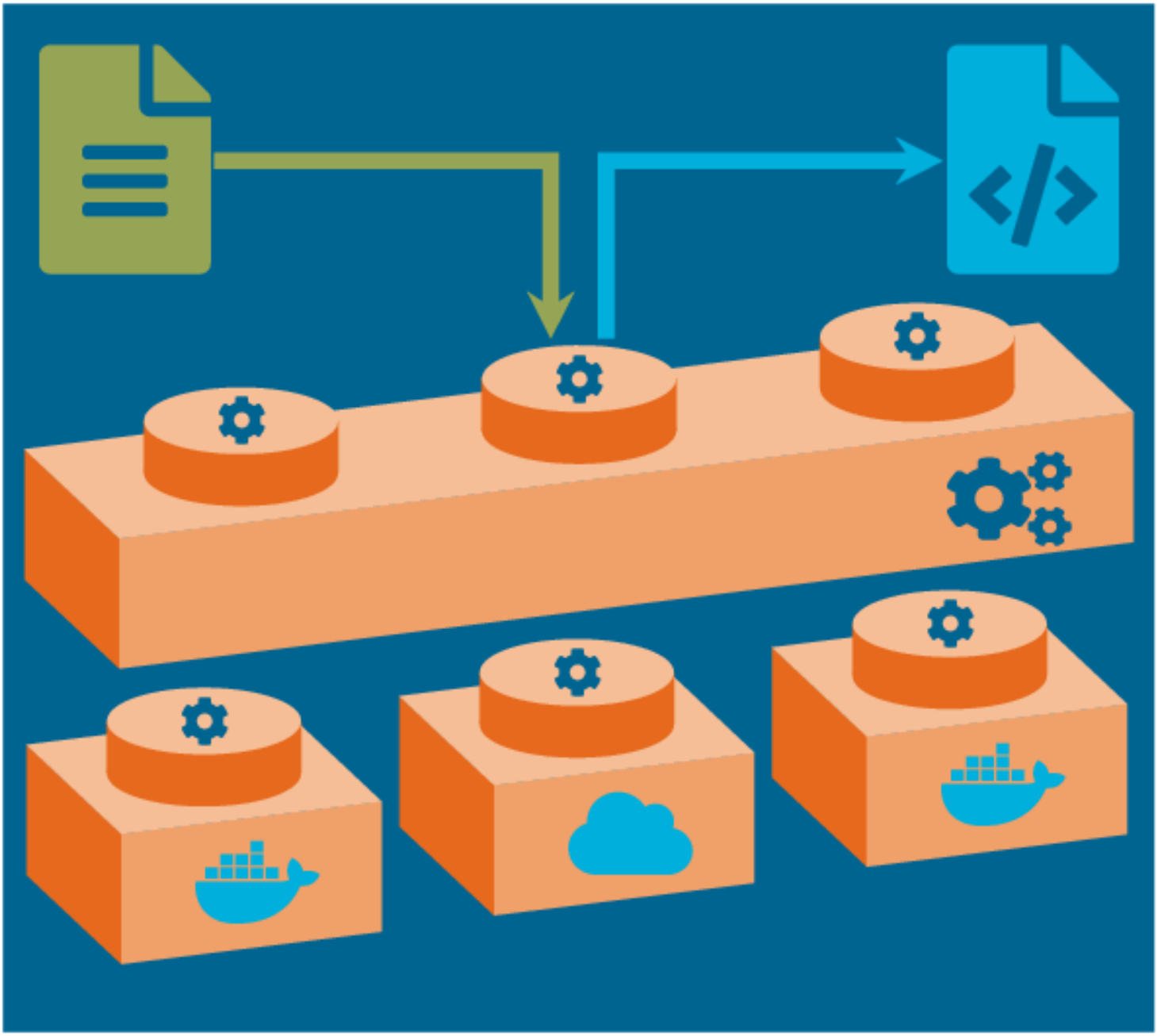
Giuseppe Abrami, Markos Genios, Filip Fitzermann, Daniel Baumartz and Alexander Mehler. (2025). “Docker Unified UIMA Interface: New perspectives for NLP on big data” SoftwareX, Volume 29, 2025, 102033, ISSN 2352-7110, [LINK]

BibTeX

```
@inproceedings{Leonhardt:et:al:2023,
  title     = {Unlocking the Heterogeneous Landscape of Big Data {NLP} with {DUUI}},
  author    = {Leonhardt, Alexander and Abrami, Giuseppe and Baumartz, Daniel and Mehler, Alex
  editor    = {Bouamor, Houda and Pino, Juan and Bali, Kalika},
  booktitle = {Findings of the Association for Computational Linguistics: EMNLP 2023},
  year      = {2023},
  address   = {Singapore},
  publisher = {Association for Computational Linguistics},
  url       = {https://aclanthology.org/2023.findings-emnlp.29},
  pages     = {385--399},
  pdf       = {https://aclanthology.org/2023.findings-emnlp.29.pdf},
  abstract  = {Automatic analysis of large corpora is a complex task, especially
               in terms of time efficiency. This complexity is increased by the
               fact that flexible, extensible text analysis requires the continuous
               integration of ever new tools. Since there are no adequate frameworks
               for these purposes in the field of NLP, and especially in the
               context of UIMA, that are not outdated or unusable for security
               reasons, we present a new approach to address the latter task:
               Docker Unified UIMA Interface (DUUI), a scalable, flexible, lightweight,
               and feature-rich framework for automatic distributed analysis
               of text corpora that leverages Big Data experience and virtualization
               with Docker. We evaluate DUUI{'}s communication approach against
```

```
                a state-of-the-art approach and demonstrate its outstanding behavior
                in terms of time efficiency, enabling the analysis of big text
                data.}
}

@article{Abrami:et:al:2025:a,
  title     = {Docker Unified UIMA Interface: New perspectives for NLP on big data},
  journal   = {SoftwareX},
  volume    = {29},
  pages     = {102033},
  year      = {2025},
  issn      = {2352-7110},
  doi       = {https://doi.org/10.1016/j.softx.2024.102033},
  url       = {https://www.sciencedirect.com/science/article/pii/S2352711024004047},
  author    = {Giuseppe Abrami and Markos Genios and Filip Fitzermann and Daniel Baumartz and
}
```



View project on GitHub Powerd by

❝ Cite § Impress

This page was generated by GitHub Pages.